

Notebook

February 8, 2026

1 Bleichenbacher's Padding Oracle

Modern variants of this attack surfaced in the recent years [The ROBOT Attack](#), but the original itself can still be found lurking in some old libraries (actually, a similar bug has just been fixed in [OpenSSL 3.0.8](#) on the 7th of February 2023). ## Basic RSA The RSA primitive ($c = Enc(m) = m^e \bmod N$; $m = Dec(c) = c^d \bmod N$) has a few issues: 1. If $m_1 = m_2$, then $c_1 = c_2$, which is a problem, since it allows to Man-in-the-Middle to derive information. (For example, we can know that the same person is using the system at this time, since they sent the same encrypted password) 2. If $e \cdot bitlen(m) < bitlen(N)$, then m^e doesn't wrap the modulus and can be easily decrypted by taking the e -th root of C

1.1 PKCS#1 v1.5

To address these issues RSA (as in company) created a specification, that became [PKCS#1 v1.5](#) (Public Key Cryptography Standards) The specification introduces formatting for encrypted (or signed) blocks (effectively, a special padding). Let's say the length of modulus N in octets (bytes) is equal to k : $k = ||N||$. The data we want to encrypt/sign is D . Then the encryption block is:

$EB = 00 || BT || PS || 00 || D$

where BT is 00 or 01 for private-key operations (signing) and 02 is for public-key operations (encryption). The encryption block is translated to integer using big-endian conversion (the first octet of EB is the most significant one) PS consists of $k - 3 - ||D||$ octets. Its contents depend on BT : 1. $BT = 00$, all octets in PS have value 00 2. $BT = 01$, all octets in PS have value FF 3. $BT = 02$, all octets in PS are pseudorandomly generated and not equal to 00

It is recommended to use $BT = 01$ for signatures and $BT = 02$ for encryptions, since 1. With $BT = 01$ D can be unpadding regardless of its contents, with $BT = 00$ the first byte of D , $D_0 = 00$ will also be unpadding, which creates a problem. 2. Both $BT = 01$ and $BT = 02$ create large integers for encryption, so all attacks depending on small D don't work

You can also notice, that the first octet of EB is 00. It is chosen so that the integer resulting from EB is always less than N .

1.2 Bleichenbacher's Padding Oracle Attack

Frankly, Dr. Matthew Green summed it up best in [What is the Random Oracle Model and why should you care? \(Part 4\)](#) : >Now, I said that almost nobody of a practical bent had a problem with PKCS. One notable exception was a bright cryptographer from Bell Labs named Daniel Bleichenbacher. >>Dr. Bleichenbacher had a real problem with the PKCS padding standards. In fact, one could say that he made it his life's mission to destroy them. This vendetta reached its zenith in 2006, when he showed how to break common implementations of PKCS signature with a pencil and paper, shortly before driving a fireworks-laden minivan into the headquarters of RSA Data Security. >>(Ok, the last part did not happen. But the rest of it did.) >>Bleichenbacher took his first crack at PKCS in CRYPTO 1998. In a surprisingly readable paper, he proposed the first practical adaptive chosen ciphertext attack against "protocols using the PKCS #1" encryption standard. Since, as I've just mentioned, the major protocol that met this description was SSL, it was a pretty big deal."

You can find the original paper here: [Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS#1](#).

Let's imagine we sent a random integer c for decryption. What is the probability that no error will surface during unpadding? The first 2 octets of EB should be $00||02$, the other octets should have at least one 00. If k is 256, then $P = \frac{1}{256^2} \cdot (1 - (\frac{255}{256})^{254}) \approx \frac{1}{104031}$ Seems way too small, but you can actually use it. The attack works as follows:

We choose the c , for which we want to find $m = c^d \bmod N$

For convenience we use $B = 2^{8(k-2)}$, $k = ||N||$. We say that ciphertext is PKCS conforming if it decrypts to a PKCS conforming plaintext.

Basically, we choose integers s , compute $c' = cs^e \bmod N$ and send them to the oracle. If c' passes the error check, then $2B \leq ms \bmod N < 3B$. By collecting enough different s we can derive m , this typically requires around 2^{20} ciphertexts, but this number varies widely. The attack can be divided into three phases: 1. Blinding the ciphertext, creating c_0 , that corresponds to unknown m_0 2. Finding small s_i , such that $m_0 s_i \bmod N$ is PKCS-conforming. For every such s_i the attacker computes intervals that must contain m_0 using previously known information 3. Starts when only one interval remains. The attacker has sufficient information about m_0 to choose s_i such that $m_0 s_i \bmod N$ is more likely to be PKCS conforming than a randomly chosen message. The size of s_i is increased gradually, narrowing the possible range of m_0 until only one possible value remains.

(Actually, when the interval is small enough, it is more efficient to just bruteforce the values locally, than continue submitting queries to the oracle)

1.3 Complete algorithm (from the [paper](#))

M_i is a set of (closed) intervals that is computed after a successful s_i has been found, such that m_0 is contained in one of the intervals of M_i ### Step 1: Blinding. Given an integer c , choose different random integers s_0 ; then check, by accessing the oracle, whether $c(s_0)^e \bmod N$ decrypts to PKCS conforming m' . For the first successful s_0 , set:

$$c_0 \leftarrow c(s_0)^e \bmod N$$

$$M_0 \leftarrow \{[2B, 3B - 1]\}$$

$$i \leftarrow 1.$$

Step 2: Searching for PKCS conforming messages **### Step 2.a:** Starting the search. If $i = 1$, then search for the smallest positive integer $s_1 \geq N/(3B)$, such that the ciphertext $c_0(s_1)^e \bmod N$ is PKCS conforming. **### Step 2.b:** Searching with more than one interval left. Otherwise, if $i > 1$ and the number of intervals in M_{i-1} is at least 2, then search for the smallest integer $s_i > s_{i-1}$, such that the ciphertext $c_0(s_i)^e \bmod N$ is PKCS conforming. **### Step 2.c:** Searching with one interval left. Otherwise, if M_{i-1} contains exactly one interval (i.e. $M_{i-1} = \{[a, b]\}$), then choose small integer values r_i, s_i such that

$$r_i \geq 2 \frac{bs_{i-1} - 2B}{N} \quad (1)$$

and

$$\frac{2B + r_i N}{b} \leq s_i < \frac{3B + r_i N}{a}, \quad (2)$$

until the ciphertext $c_0(s_i)^e \bmod N$ is PKCS conforming.

1.3.1 Step 3: Narrowing the set of solutions.

After s_i has been found, the set M_i is computed as

$$M_i \leftarrow \bigcup_{(a,b,r)} \left\{ \left[\max \left(a, \left\lceil \frac{2B + rN}{s_i} \right\rceil \right), \min \left(b, \left\lfloor \frac{3B - 1 + rN}{s_i} \right\rfloor \right) \right] \right\} \quad (3)$$

for all $[a, b] \in M_{i-1}$ and

$$\frac{as_i - 3B + 1}{N} \leq r \leq \frac{bs_i - 2B}{N}$$

Step 4: Computing the solution. If M_i contains only one interval of length 1 (i.e., $M_i = \{[a, a]\}$), then set $m \leftarrow a(s_0)^{-1} \bmod N$, and return m as solution of $m = c^d \bmod N$. Otherwise, set $i \leftarrow i + 1$ and go to step 2.

1.3.2 Remarks.

Step 1 can be skipped if c is already PKCS conforming (i.e., when c is an encrypted message). In that case, we set $s_0 \leftarrow 1$. However, step 1 is always necessary for computing a signature, even if we do not wish to get a blind signature.

In step 2.a, we start with $s_1 = \lceil N/(3B) \rceil$, because, for smaller values $m_0 s_1$ is never PKCS conforming.

We use condition (1) because we want to divide the remaining interval in each iteration roughly in half. Also, you can use exhaustive search during the later stages of the attack, when the message space is sufficiently small.

1.4 Task

You are given a server with Bleichenbacher's padding oracle. Implement the attack and decrypt the ciphertext (ciphertext is PKCS conforming). First you have a local version of the server to speed up development and testing. Test out your solution on this version. Try to use the **get_pkcs_conforming_mult** method as it makes interaction with the server faster. You just need to give it a list of intergers or bytes.

```
[ ]: from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_v1_5
from Crypto.Util.number import bytes_to_long, long_to_bytes
from Crypto.Random import get_random_bytes
class LocalVulnServerClient:
    def __init__(self, show=True):
        """Initialization, generate key"""
        self.key=RSA.generate(2048)
        self.cipher=PKCS1_v1_5.new(self.key)
        self.flag_message="FLAG FOR TESTING"
        self.sentinel=get_random_bytes(16)
    def get_public_key(self):
        return (self.key.e, self.key.n)
    def get_ciphertext(self):
        return bytes_to_long(self.cipher.encrypt(self.flag_message.
        encode()))
    def get_pkcs_conforming(self, ciphertext, show=False):
        if isinstance(ciphertext, int):
            ciphertext=long_to_bytes(ciphertext)
        if isinstance(ciphertext, bytes):
            if len(ciphertext)>256:
                print ('Ciphertext too long')
                return None
            if len (ciphertext)<256:
                ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
            result=self.cipher.decrypt(ciphertext, self.sentinel)
            return result!=self.sentinel
    def get_pkcs_conforming_mult(self, ciphertexts, show=False):
        """Check several ciphertexts for PKCS conformity"""
        results=[]
        for ciphertext in ciphertexts:
            if isinstance(ciphertext, int):
                ciphertext=long_to_bytes(ciphertext)
            if isinstance(ciphertext, bytes):
                if len(ciphertext)>256:
                    print ('Ciphertext too long')
                    return None
                if len (ciphertext)<256:
                    ciphertext=bytes([0]*(256-
len(ciphertext)))+ciphertext
```

```

        result=self.cipher.decrypt(ciphertext,self.sentinel)
        results.append(result!=self.sentinel)
    return results

```

```

[ ]: vs=LocalVulnServerClient()
    (e,N)=vs.get_public_key()
    c=vs.get_ciphertext()
    print(c)
    print (vs.get_pkcs_conforming(c,False))
    print (vs.get_pkcs_conforming(2))

```

Your code goes here

Now that you've solved it for the local version, let's break the server one

```

[ ]: import socket
    import re
    from Crypto.Util.number import inverse,long_to_bytes,bytes_to_long
    class VulnServerClient:
        def __init__(self,show=True):
            """Initialization, connecting to server"""
            self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
            self.s.connect(('cryptotraining.zone',1344))
            if show:
                print (self.recv_until().decode())
        def recv_until(self,symb=b'\n>'):
            """Receive messages from server, by default till new prompt"""
            data=b''
            while True:
                data+=self.s.recv(1)
                if data[-len(symb):]==symb:
                    break
            return data
        def get_public_key(self,show=True):
            """Receive public key from the server"""
            self.s.sendall('public\n'.encode())
            response=self.recv_until().decode()
            if show:
                print (response)
            e=int(re.search('(?!<=e: )\d+',response).group(0))
            N=int(re.search('(?!<=N: )\d+',response).group(0))
            return (e,N)

        def get_ciphertext(self,show=True):
            """Receive ciphertext from the server"""

```

```

self.s.sendall('ciphertext\n'.encode())
response=self.recv_until().decode()
if show:
    print (response)
c=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext: )[0-9a-f]+' ,response).group(0)))
return c

def get_pkcs_conforming(self, ciphertext, show=True):
    """Check if the ciphertext is PKCS conforming"""
    if isinstance(ciphertext,int):
        ciphertext=long_to_bytes(ciphertext)
    if len(ciphertext)>256:
        print ('Ciphertext too long')
        return None
    if len (ciphertext)<256:
        ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
    ciphertext_hex=ciphertext.hex()

    self.s.sendall(f'unpad {ciphertext_hex}\n'.encode())

    response=self.recv_until().decode()
    if show:
        print (response)
    return response.find('error')== -1

def get_pkcs_conforming_mult(self, ciphertexts,show=False):
    """Check several ciphertexts for PKCS conformity"""
    newc=[]
    for x in ciphertexts:
        if isinstance(x,int):
            ciphertext=long_to_bytes(x)
        else:
            ciphertext=x
        if len(ciphertext)>256:
            print ('Ciphertext too long')
            return None
        if len (ciphertext)<256:
            ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
        ciphertext_hex=ciphertext.hex()
        newc.append(ciphertext_hex)
    self.s.sendall((' \n'.join([f'unpad {chex}' for chex in_
↵newc])+' \n').encode())

    results=[]
    for i in range(len(newc)):
        response=self.recv_until().decode()

```

```
        if show:
            print (response)
        results.append(response.find('error')== -1)
    return results
```

```
def __del__(self):
    self.s.close()
```

```
[ ]: vs=VulnServerClient()
      (e,N)=vs.get_public_key()
      c=vs.get_ciphertext()
      print(c)
      print (vs.get_pkcs_conforming(c, False))
```

```
[ ]: # Your code goes here
```